Time as Intervals, Not Instants: A Practical Implementation of Allen's Interval Algebra Over ISO 8601

Kip Cole
Independent

Abstract

Mainstream programming languages model temporal values as scalar instants: a `Date` is a point, a `DateTime` is a point, and a partial value such as "June 2026" has no canonical representation at all. Yet the temporal values that humans, calendars, and business logic actually manipulate are spans on the timeline — the year 2026, the afternoon of the 15th, an archaeological period, a meeting. Tempo is an Elixir library that reverses the polarity. Every expression — a value, an interval, a recurrence, a set — denotes an ordered sequence of bounded half-open spans; a value written to a resolution denotes one unit of it; set-theoretic combination is closed over the one type; and comparison is defined by Allen's thirteen relations rather than by scalar ordering. A written interval is read as it is said aloud: an end written to a calendar unit is included, an end written to a clock unit is a boundary. The result eliminates a class of off-by-one and "end-of-day" bugs by construction, gives partial ISO 8601 values (`2026`, `2026-06`, `2026-06-XX`) an unambiguous denotational semantics, and lifts Allen's algebra from a textbook abstraction to an everyday predicate vocabulary (`overlaps?`, `within?`, `adjacent?`). This extended abstract describes the model as nine laws, the frames on which spans are placed, the reading of written ends, and the design of set operations over spans and sets of them, and reports on the implementation written to the laws.

2012 ACM Subject Classification Information systems → Temporal data; Software and its engineering → Domain specific languages; Theory of computation → Modal and temporal logics

Keywords and phrases Temporal intervals, Allen's interval algebra, ISO 8601, calendrical systems, partial dates, set operations on time, domain-specific languages

Digital Object Identifier 10.4230/OASIcs...

Related Version Source repository: `https://github.com/elixir-tempo/tempo`

1 Motivation: the instant–span impedance mismatch

Every mainstream temporal library — Python's `datetime`, Java's `java.time`, JavaScript's `Temporal`, Elixir's own `Date`/`Time`/`DateTime` — shares a design choice that goes largely unexamined: a temporal value is a *scalar instant* on the timeline. A day is the midnight at its start, a month is the midnight on its first, a year is the midnight on January 1. The span that humans actually mean when they say "June 2026" is materialised, if at all, by ad-hoc helpers (`start_of_month`, `end_of_day`) that each project re-implements with its own conventions about inclusivity, time zones, and DST.

This instant-first model has three persistent consequences:

Ambiguity at the type level.

The expression `Date.new(2026, 6, 15)` denotes a 24-hour span in business prose and an instant in the type system. The disagreement is resolved informally, per call site, and is a recurring source of off-by-one bugs at month, year, and DST boundaries. Tiwari et al.'s recent empirical study of 151 date/time bugs in open-source Python projects identifies time-zone-related mistakes as the largest single category, with most bugs traceable to misconceptions



© Kip Cole;
licensed under Creative Commons License CC-BY 4.0



about library API conventions and edge-case behaviour [10]; Liva's earlier work on Java programs reaches a structurally similar conclusion [11].

No representation for partial dates.

A user who writes "2022" or "the 1560s" or "approximately June 2026" has produced a perfectly meaningful temporal value, but the type system has no home for it. Solutions range from sentinel values (`Date.new(2022, 1, 1)` stands in for "the year") to parallel type hierarchies (EDTF [8]) that exist outside the standard library, or to dedicated calendric type systems such as Bry and Spranger's CaTTS for Semantic Web query languages [12]. The fragmentation is structural, not incidental.

Comparison is reduced to scalar ordering.

Two dates are either `<`, `=`, or `>`. The richer relational structure that Allen identified in 1983 [1] — *precedes*, *meets*, *overlaps*, *starts*, *during*, *finishes*, *equals*, and their inverses — is invisible at the type level, and reimplemented case-by-case in business logic that interrogates whether a meeting "overlaps" the workday or a contract "contains" a billing period.

Tempo's design is the observation that all three problems dissolve simultaneously if intervals are made the primitive type. There are no points: an apparent "instant" is the interval of one unit at the finest declared resolution, in the same spirit as the time ontology $T_{bounded_meeting}$ of Grüninger and Li [13], in which intervals are the only entities in the domain and Allen's relations are derivable from a single primitive (*meets*).

Empirical coverage.

Against the 151 Python bugs catalogued by Tiwari et al., the design choices described in the remainder of this paper address every in-scope bug class: 43 (28%) are *avoided by construction* (the naïve / aware datetime split has no Tempo analogue), 56 (37%) are *directly addressed* by explicit machinery (zone propagation through set operations, refusal of a reading the clock skips and IXDTF-offset fold disambiguation per RFC 9557 §4.5, calendar-aware arithmetic, half-open materialisation, microsecond-precision durations), 28 (19%) are partial, and 24 (16%) are out of scope (Python deprecations and third-party-library logic such as holiday tables). *Zero* bugs are classified as architectural gaps. The classification was made against the library's first implementation; the rewrite described in Section 6 kept every mechanism it relies on. The per-bug classification is provided as supplementary material at the repository.

2 Expressions, spans and frames

Tempo's model is nine laws, and the implementation is written to them and to nothing else. The first is the one from which the rest follow: an *expression* — a value such as `2026-06`, an interval such as `2026-06-01/2026-06-20`, a recurrence such as `R5/2026-06-15/P1D`, a set of them — *denotes an ordered sequence of half-open spans*, $spans(E)$, possibly empty, possibly infinite, and every operation on an expression is defined on $spans(E)$. A duration denotes a length and is not an expression. One data structure holds every expression, with its form as a tagged sum (a value, a span, a recurrence, an all-of set, a one-of set, a conditional member), its calendar, its zone and the caller's metadata; there is no separate interval type and no separate set type.

A span is $[first, last)$, a pair of positions. Internally this is the standard finite model of $T_{bounded_meeting}$ characterised by Grüninger and Li [13] (their Theorem 15): vertices of the

directed graph $\mathfrak{M}^{bounded_meeting}$ correspond to intervals, edges to the *meets* relation, and structures up to isomorphism faithfully capture the ontology's models.

Resolution is meaning.

A value written to a resolution denotes one unit of it, $[v, next_r(v))$:

`2026`	≡	[`2026-01-01`, `2027-01-01`)
`2026-01`	≡	[`2026-01-01`, `2026-02-01`)
`2026-01-15`	≡	[`2026-01-15`, `2026-01-16`)
`2026-01-15T10`	≡	[`2026-01-15T10:00`, `2026-01-15T11:00`)

The partial-date problem disappears: `2026` is not an under-specified instant, it is a definite interval of one year (the distinction is semantic, not a claim about measurement precision or accuracy). The "last day of the month" problem disappears: the upper bound of `2026-01` is `2026-02-01`, computed from the calendar arithmetic of the target calendar (Gregorian, Hebrew, Islamic Civil, ...), and is correct by construction in leap years and across month-length variation. The written units are kept as written; nothing drills into them, and a value and the interval of the same extent are different expressions that enumerate differently (a month enumerates its days; the interval `2026-01/2026-01` enumerates months, one of them).

A note on terminology: *resolution* as used throughout this paper is a property of an input value (`2026` carries year-resolution, `2026-06-15` carries day-resolution) used for materialisation, default iteration, and display. It is *not* the granularity of Bettini, Wang and Jajodia [9] (a lattice of partitions of the time line that participates in Allen comparisons), and it does *not* appear in $T_{bounded_meeting}$, where intervals are individuated solely by their position in the *meets*-structure. Once an interval is materialised to its $[first, last)$ form, resolution plays no further role in Allen comparisons or in set-theoretic operations. The word is also distinct from *precision* and *accuracy* in the measurement-science sense: Tempo makes no claim about the precision of a value's construction, only about the extent it denotes.

Written ends are read as they are said.

An interval written with two ends, `a/b` in ISO 8601, runs from the start of `a`. Where it runs to depends on what `b` names. A date names a period of the calendar, and "2026 to 2028" and "1 to 3 June" include their last day in speech, in EDTF (`1964/2008` is "beginning sometime in 1964 and ending sometime in 2008") and in ISO 8601-2 Annex A. A time of day names a reading of the clock, and a reading is a boundary: "nine to five" ends when the clock shows five, not a minute later. So an end written to a *calendar* unit (a year, a month, a week, a date) is read *through*, and an end written to a *clock* unit (an hour, a minute, a second) is read as a boundary:

`2026/2028`	[`2026-01-01`, `2029-01-01`)	three years
`2026-06-01/2026-06-03`	[`06-01`, `06-04`)	three days
`T09/T17`	[`09:00`, `17:00`)	eight hours
`2026-06-15/2026-06-15T17`	[`06-15T00:00`, `06-15T17:00`)	seventeen hours
`2026-06-15T10/2026-06-16`	from 10:00 on the 15th through the 16th	thirty-eight hours

The reader makes the half-open bound and the writer writes a span back as it was read, so text round-trips. The rule needs nothing of a calendar, only the class of the unit, and it lets the half-open model stand without a notation for an exclusive end, which ISO 8601 does not have. It is the one semantic choice in the library with no direct precedent in the formalisms it draws on, and Section 7 puts it to the symposium.

Frames.

A span lies on one of three frames. A value with a year and a zone, or a written offset, lies on the *UTC line*; a value with a year and neither lies on the *floating line*; a value with no year (`T10:30`, `-06-15`, a Monday) lies on a *cycle*, the day for a time of day, the week for a weekday, the year for a month and day. A position is an integer: a microsecond tick from a fixed origin on a line, a tick from the cycle's start on a cycle. Two spans relate, compare or combine only on one frame; a floating value beside a zoned one, or a time of day beside a date, is a named error and never a guess. A value on a cycle is, in the ontology's terms, not an individual but an open expression with the year free; the `:within` option, and the operations `at/2`, `on/2` and `in_zone/2`, bind it, placing it on each period of a window, and the result is the intervalised individual $T_{bounded_meeting}$ talks about. Set operations on a cycle cut a span that runs through the cycle's end (`T22/T02`) for the sweep and join it after, so the algebra is the same on all three frames. The ontology is silent on schemas; frames, like resolution, sit in the engineering layer above it.

Half-open by design.

The half-open $[first, last)$ convention is not notational shorthand; it is enforced at every level of the API. Adjacent intervals concatenate cleanly: $[a, b) \cup [b, c) = [a, c)$ with neither overlap nor gap. Iteration yields a sequence whose union is exactly the parent interval. No span has zero extent, in line with $T_{bounded_meeting}$'s exclusion of degenerate intervals from the domain. Library code that treats the stored upper bound as inclusive is, by the design contract, a defect. Alternative timestamp representations have been explored at TIME — notably Dyreson and Ahsan's log-segmented timestamps [14, 15], which decompose a period into power-of-two segments so that standard B^+-tree indexes can support sequenced and nonsequenced queries on an unmodified DBMS. The two designs occupy different points in the same space: log-segmented timestamps optimise query evaluation in a relational engine; Tempo's half-open spans optimise expressiveness and uniformity at the application layer.

3 Allen's algebra as the predicate vocabulary

Tempo exposes Allen's thirteen relations [1] directly as the result of its core comparison primitive. The first-order ontology underlying the algebra is characterised by Grüninger and Li [13], who identify the time ontology logically synonymous with Allen's interval algebra and prove a representation theorem for its models; Tempo is, in effect, an ergonomic software realisation of that ontology over the calendar-aware datetime values defined by ISO 8601. The thirteen relations are returned directly from the relation primitive:

```
Tempo.relation(a, b) ::
  :precedes | :meets | :overlaps | :finished_by |
  :contains | :starts | :equals |
  :started_by | :during | :finishes | :overlapped_by |
  :met_by | :preceded_by
```

The relation is read from four integer comparisons of the two spans' positions (start against start, end against end, and each start against the other's end); every pair of spans stands in exactly one relation.

Beyond the relations themselves, the library ships Allen's *composition* table — the transitivity result of [1] Fig. 4 — as a first-class operation, `Tempo.Allen.compose/2`: given

$A\ r_1\ B$ and $B\ r_2\ C$, it returns the set of relations that can hold between A and C. The 169 cells are not transcribed: they are derived when the library compiles, by placing three intervals every way on eight points (six endpoints need at most six distinct values, so the enumeration is complete) and classifying each pair with the same four comparisons, and then held to the published table by test. The classifier and the composition table therefore cannot disagree.

Composition drives a propagation algorithm. Lifting it from single relations to *sets* of them gives the operations a qualitative constraint network is built from — the converse of a set, the narrowing of one set by another, and set composition — and Allen's path-consistency step is then their composite, $R(i,k) \leftarrow R(i,k) \cap (R(i,j) \circ R(j,k))$, iterated to a fixpoint. This is what lets a companion package, `tempo_network`, reason about intervals that are *not* grounded: a period known only as "before or after the rebuild, never during" is the set $\{precedes, preceded_by\}$, and propagation derives what that implies elsewhere in the network. Pruning is sound — every relation removed is genuinely impossible — but path consistency does not decide satisfiability, which is NP-complete for the full algebra [2, 4]; the library says so at the call site rather than letting a clean pass be read as a proof.

This is rarely the right surface for application code, however — business questions tend to name unions of Allen relations. "Does the meeting fall within the workday?" is the set $\{equals, starts, during, finishes\}$; "does this period touch that one?" is the complement of $\{precedes, preceded_by\}$. Hand-rolled inline matching on relation lists is verbose and obscures the intent, so the library promotes the common unions to first-class predicates — `within?/2`, `overlaps?/2`, `adjacent?/2`, `contains?/2` — each defined as a specific subset of Allen's relations, held in one table, and each readable aloud as the English question it answers.

The predicate vocabulary is not decorative. It is a design discipline: when an example in the cookbook reaches for a relation list or a manual endpoint comparison, that is taken as evidence that the abstraction is missing, and the predicate is added before the example is written. The same discipline applies to durations (`at_least?`, `shorter_than?`) and to interval-set queries (`covered?`, `disjoint?`). The library's public surface is, deliberately, an English-prose description of Allen's algebra over ISO 8601 values.

4 Position among temporal formalisms

The choice to make intervals primitive places Tempo on one side of a long-standing divide. Vilain and Kautz [2] take *points* as primitive: an interval is an ordered pair of endpoint points, and an interval relation reduces to a conjunction of point-relations $\{<, =, >\}$ among the four endpoints. The reward is tractability — reasoning in the point algebra is polynomial, whereas constraint reasoning over the full interval algebra is intractable in general [2, 4]. Allen and Hayes [3] take the opposite stance, axiomatising the algebra from the single primitive *meets* and reconstructing points only as derived entities — *moments* (intervals with no proper subinterval) and the meeting-places between intervals. Grüninger and Li's $T_{bounded_meeting}$ [13] continues this interval-only lineage, excluding degenerate intervals and points from the domain entirely; this is the ontology Tempo adopts.

Tempo's contribution is best read as a synthesis across this divide. *Ontologically* it is interval-only: there are no instants, degenerate intervals are excluded, and the half-open seam realises Hayes' open-connected model rather than Allen's closed intervals. *Computationally*, however, every end is an integer position on one frame, and the engine orders spans by their ends — precisely the Vilain–Kautz reduction, used as an engine beneath an interval ontology. Two consequences follow. First, because Tempo's spans are always *fully grounded*

(concrete ends, never disjunctive qualitative constraints), it never enters the NP-hard regime that motivated the point algebra: relation queries are four integer comparisons rather than constraint propagation, and every set operation is one sweep over sorted ends. Calendars and zones are consulted only to compute positions, once per end. Second, the "instant" is recovered not as a primitive point but as the interval of one unit at the value's *finest declared resolution* — a *moment* in Allen and Hayes' sense, but relativised to representational precision: a second-resolution datetime is a one-second interval, a microsecond value a one-microsecond interval. This resolution-indexed atomicity, tying interval granularity to ISO 8601 syntax rather than to an absolute atom, and the reading of written ends by the class of their unit, are the two elements of the model without a direct precedent in the formalisms it draws on.

5 Set operations on spans and sets of them

Closing the library under set-theoretic operations is where the model earns its keep. The intuition is straightforward — any two expressions combine into a (possibly disjoint, possibly empty) set of spans — but the engineering requires one answer to the questions of resolution, of frames, of calendars and of what a result *is* when its operands were events with identities of their own.

One sweep.

Every instant-level operation is the same sweep over the operands' sorted ends with a different question of how many members of each operand cover a point: `intersection/3` keeps a point covered by both, `difference/3` a point covered by the first and not the second, `symmetric_difference/3` a point covered by exactly one, and the n-ary "at least k members" query a point covered by k or more. A result begins where the answer turns true and ends where it turns false, so a result's ends are always operands' ends, and are written back as those operands wrote them, at their resolution: the difference of a month and a day ends on a day. Every operation returns a set; a set that happens to have one member is still a set, which keeps the operations closed and composable without a case split at the call site.

Members and covered time.

The textbook operations answer about *covered time*. Application code as often asks about *members*: which meetings hit this window, which workdays are not holidays. Tempo keeps the two apart. `union/3` preserves members (two meetings that cover the same hour stay two meetings, each with its metadata) and the coalesced form is one explicit operation away; the other operations trim to covered time, and each result is credited to the operand members that cover it, so that a holiday's name survives the difference that removed the weekend from it. Each instant-level operation has a member-preserving companion (`members_overlapping/3`, `members_outside/3`, `members_in_exactly_one/3`), and the canonical form of a set — sorted by start, coalesced where members touch — is one operation, `coalesce/2`, which realises the Sum Axiom of $T_{bounded_meeting}$ (axiom 15 in [13]): three chain-meeting intervals $meets(i, j) \wedge meets(j, k) \wedge meets(k, m)$ are summed into the single interval that runs from i's start to m's end.

One expression, one calendar, one zone.

The calendar and the zone are properties of a whole expression, written once after it (`2026-06-15T10:30/T12:00[Europe/Paris][u-ca=hebrew]`), never of one end or one mem-

ber: an interval whose two ends are written in two calendars, or two named zones, is refused at construction. Across expressions the rule is explicit conversion. A set operation brings its second operand to the calendar of its first; two zoned operands meet on the UTC line and the result is written in the first operand's zone; a floating operand beside a zoned one is a named error unless the caller places it. The result's calendar and zone are therefore the first operand's, which preserves the caller's intent for display, and what cannot be written in one calendar and one zone is an error rather than another value. An earlier version of this design projected every operand to a common line and let the first operand win silently; the stricter rule adopted the alternative — refuse heterogeneous frames, convert calendars explicitly — that the authors had invited the symposium to weigh, and it is one of the points the talk would revisit.

Cross-calendar positions.

A date is placed on the day scale as a proleptic Gregorian day number, the fixed date of Reingold and Dershowitz, which every calendar converts to and from without a path through another calendar; a reading of the clock is placed as a microsecond tick on the UTC line through the zone. The word "instant" is avoided because Grüninger and Li's ontology excludes points from its domain; the projection is a computational device, an integer used to order ends on one frame.

Daylight-saving transitions.

Because every span is read through its zone to positions on the UTC line, transitions fall out of the walk rather than being handled by hand. A day in Paris on which the clock springs forward has twenty-three hours because its two ends are twenty-three hours apart; an hour the clock skips in part is as long as the clock shows it; a day a zone leaves out (Samoa's 30 December 2011) is no day of a walk through that December. Two cases are settled at construction, so that set operations on positions need no further disambiguation. A wall time that falls in a DST *gap* (the non-existent hour at spring-forward) is refused as a value: a `Tempo.ZoneError` whose reason is `:gap`, which is a stronger guarantee than addressing the bug, because the malformed value cannot be constructed. A wall time that falls in a DST *fold* (the doubled hour at fall-back) is disambiguated by an explicit offset in the IXDTF suffix, per RFC 9557 §4.5: the value `2024-11-03T01:30:00-04:00[America/New_York]` unambiguously picks the pre-fall-back (EDT) showing and `2024-11-03T01:30:00-05:00[America/New_York]` picks the post-fall-back (EST) showing. The disambiguator is part of the string representation and round-trips through serialisation — a property that distinguishes this design from meta-attribute schemes (such as Python's `fold` parameter) in which the choice is carried out-of-band and lost on serialisation.

Expressions with no end.

A recurrence with no end is a set with infinitely many members, and the operations answer it exactly or not at all: an intersection with a bounded operand lies within that operand and is taken there; a union of two endless operands is an endless set in time order, read lazily; an intersection of two endless operands, which may be searched for ever, is refused unless the caller gives a window.

6 Implementation

Tempo is a working library, and the model above is its second implementation. The first grew to 82,875 lines by accretion and carried the same handful of concepts in parallel forms: a review in October 2026 counted ten walks, six sweeps, five copies of the Allen table, four selection engines and two arithmetics. The second was written to the nine laws in one pass and is 26,862 lines, of which 14,524 are code and 7,806 are the engine (positions and frames, the calendar and zone contracts, expansion, the walk, the sweep, the classifier, selection, arithmetic). Each concept is implemented once, and the calendar is reached through one contract of eighteen questions that never asks what kind of calendar it is talking to, so that the Gregorian, Hebrew, Islamic, Chinese, Julian and fiscal calendars join by the same door.

The implementation is held to oracles written apart from it: an independent expander of 192 recurrence rules from three starts, itself held to RFC 5545's own examples; a check of every set operation on every pair of sets of up to two members between five points by integer arithmetic on the standard library's dates; Allen's composition table held to what three spans give; twelve zones over twenty-one days each held to the standard library's clock; and a totality check that every operation on every expression of a corpus is an answer or a named error. Against the first implementation, reading text is thirty times faster, writing and walking a fifth faster, relating two values about the same (1.4 μs), a set operation of two sets of a hundred members twice as slow, and comparing two values, which the first did by their fields, seven times slower at 1.5 μs. Recurrence rules, which the first implementation expanded through a fast path for weekdays beside its resolver, are four to six times faster for the weekly and monthly rules of the benchmark and within a fifth of it for daily rules, once a plain rule is compiled to a test of a day number and taken in front of the one engine, which remains the meaning of every rule and holds the compiled form to itself by the same measures. What remains, by the implementation's own profile, is the placing of values, which the engine holds as their written units; the model is unchanged by any of it.

7 Discussion and open questions

This submission's contribution is a *design* contribution rather than a *result* contribution, and the talk would aim to make the design choices legible to an audience whose research is the theory the library implements. Four questions seem particularly worth the TIME audience's attention:

1. **Written ends.** The rule of Section 2, that an end written to a calendar unit is included and an end written to a clock unit is a boundary, reconciles the inclusive reading of EDTF and ISO 8601-2 Annex A with a half-open model by appeal to how the two kinds of end are said aloud. It is exact and it round-trips, but it makes `2026-06-15T10/2026-06-16` thirty-eight hours where a reader used to boundaries expects fourteen. Is a rule keyed to the class of the unit the right reconciliation, or should the standard carry a notation for an exclusive end?
2. **Granularity and resolution.** Tempo's one-unit-of-the-written-resolution rule for values is operationally simple but does not reference the more general treatment of temporal granularity in the database literature [9] nor the calendric type constraints in CaTTS [12]. Is the simple rule a sufficient model in practice, or does it accumulate edge cases as the calendar set grows (lunar, sidereal, fiscal)? The three frames, and the cycle in particular, are the places the question is felt.

3. **Allen's algebra over sets.** The thirteen relations, and the $T_{bounded_meeting}$ ontology that underlies them, are defined on convex intervals only [13]. Their lifting to sets admits several reasonable definitions, and the library now offers three: the covered-time lifting through the predicates (*within* is an empty difference, *overlaps* a non-empty intersection), the existential lifting through the member-preserving companions, and the set-of-relations lifting through `relation_matrix/2`, which returns the relation holding between every pair of members. Which should be the *default* is not obvious, and a principled extension of the ontology to non-convex sets would settle it.
4. **Cross-calendar comparison.** Comparing a Hebrew date with a Gregorian one resolves cleanly when both are placed on the same day scale, but the scale is itself a modelling choice (TAI? UT1? a calendar-specific civil time?). Tempo's choice — the proleptic Gregorian day number for dates and UTC for readings of the clock, with calendars converted explicitly and frames never mixed — is pragmatic; a principled framework would be a worthwhile contribution from the symposium back to the library.

The wider goal is to demonstrate, by existence proof, that the theoretical structure of temporal reasoning translates into an ordinary application library — one that an engineer writes `Tempo.overlaps?(a, b)` into without thinking of it as a research artefact. The same primitive reaches beyond set algebra: a thin constraint layer reduces a network of partially known intervals to a Simple Temporal Problem [16], reproducing the chronological-network model of Levy et al. [17] — realised in the ChronoLog tool [18] — as a library feature rather than a bespoke application.[1] The talk would walk through the design, the choices made, the choices still open, and invite the symposium to sharpen them.

References

1 James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi:10.1145/182.358434

2 Marc B. Vilain and Henry A. Kautz. Constraint propagation algorithms for temporal reasoning. In *Proceedings of the 5th National Conference on Artificial Intelligence (AAAI-86)*, pages 377–382, 1986.

3 James F. Allen and Patrick J. Hayes. Moments and points in an interval-based temporal logic. *Computational Intelligence*, 5(3):225–238, 1989. doi:10.1111/j.1467-8640.1989.tb00329.x

4 Bernhard Nebel and Hans-Jürgen Bürckert. Reasoning about temporal relations: A maximal tractable subclass of Allen's interval algebra. *Journal of the ACM*, 42(1):43–66, 1995. doi:10.1145/200836.200848

5 International Organization for Standardization. *ISO 8601-1:2019 — Date and time — Representations for information interchange — Part 1: Basic rules*, 2019.

6 International Organization for Standardization. *ISO 8601-2:2019 — Date and time — Representations for information interchange — Part 2: Extensions*, 2019.

7 Ujjwal Sharma and Carsten Bormann. *Date and Time on the Internet: Timestamps with Additional Information*. RFC 9557, IETF, April 2024. doi:10.17487/RFC9557

8 Library of Congress. *Extended Date/Time Format (EDTF) Specification*, 2019. https://www.loc.gov/standards/datetime/

9 Claudio Bettini, Sushil Jajodia, and Sean X. Wang. *Time Granularities in Databases, Data Mining, and Temporal Reasoning*. Springer, 2000.

[1] Developed in a companion paper, *Chronological Networks as a Library Primitive: ChronoLog over an Interval-Native Date/Time Stack*.

10 Shrey Tiwari, Serena Chen, Alexander Joukov, Peter Vandervelde, Ao Li, and Rohan Padhye. It's about time: An empirical study of date and time bugs in open-source Python software. In *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR '25)*. IEEE/ACM, 2025. Distinguished Paper Award. https://rohan.padhye.org/files/datetimebugs-msr25.pdf

11 Giovanni Liva. *Modeling Time in Java Programs for Automatic Error Detection*. PhD thesis, Alpen-Adria-Universität Klagenfurt, 2018.

12 François Bry and Stephanie Spranger. Towards a multi-calendar temporal type system for (Semantic) Web query languages. In *Principles and Practice of Semantic Web Reasoning (PPSWR 2004)*, volume 3208 of *Lecture Notes in Computer Science*, pages 90–104. Springer, 2004. doi:10.1007/978-3-540-30122-6_8

13 Michael Grüninger and Zhuojun Li. The time ontology of Allen's interval algebra. In *24th International Symposium on Temporal Representation and Reasoning (TIME 2017)*, volume 90 of *LIPIcs*, pages 16:1–16:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017. doi:10.4230/LIPIcs.TIME.2017.16

14 Curtis E. Dyreson and M. A. Manazir Ahsan. Achieving a sequenced, relational query language with log-segmented timestamps. In *28th International Symposium on Temporal Representation and Reasoning (TIME 2021)*, volume 206 of *LIPIcs*, pages 14:1–14:13. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.TIME.2021.14

15 Curtis E. Dyreson. Optimization of nonsequenced queries using log-segmented timestamps. In *30th International Symposium on Temporal Representation and Reasoning (TIME 2023)*, volume 278 of *LIPIcs*, pages 13:1–13:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPIcs.TIME.2023.13

16 Rina Dechter, Itay Meiri, and Judea Pearl. Temporal constraint networks. *Artificial Intelligence*, 49(1–3):61–95, 1991. doi:10.1016/0004-3702(91)90006-6

17 Eythan Levy, Gilles Geeraerts, Frédéric Pluquet, Eli Piasetzky, and Alexander Fantalkin. Chronological networks in archaeology: A formalised scheme. *Journal of Archaeological Science*, 124:105225, 2020. doi:10.1016/j.jas.2020.105225

18 ChronoLog. http://chrono.ulb.be.